GPUs: From Graphics to AI

Part 3: The GPU as an AI Architecture

MAC units and systolic arrays in SystemVerilog

Hardware Description Languages

Review from Part 2

Kernels are written for **one** thread

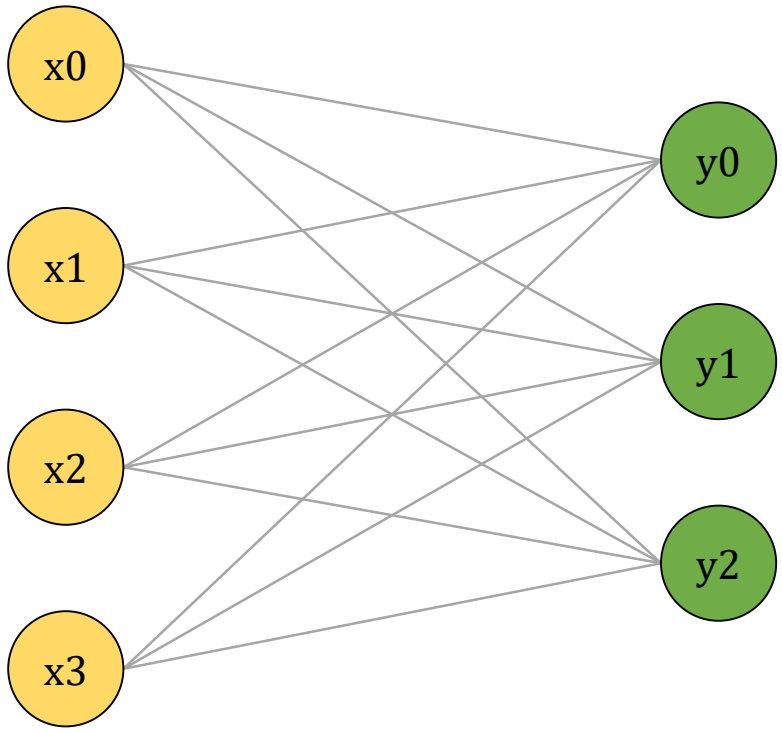
Warps of 32 threads share one instruction

Matrix multiply: $\mathbf{2\ N^3}$ operations

Tiling reuses data from shared memory

cuBLAS was much faster than our kernel. **Why?**

A Neural Network Layer



Every line is a **weight**

Each output:

weight each input

add them up

$$\mathbf{y = W x + b}$$

A whole batch of inputs at once: $Y = W X$ - a matrix multiply

It All Comes Down to MAC

Multiply-accumulate:

acc <= acc + a × b

One layer, 4096 × 4096 weights, batch of 4096:

$4096^3 \approx$ **69 billion** MACs

A large model has hundreds of such layers

Make the MAC cheap and plentiful, and you win

Arithmetic Intensity

Intensity = operations / bytes moved

Vector add, FP32:

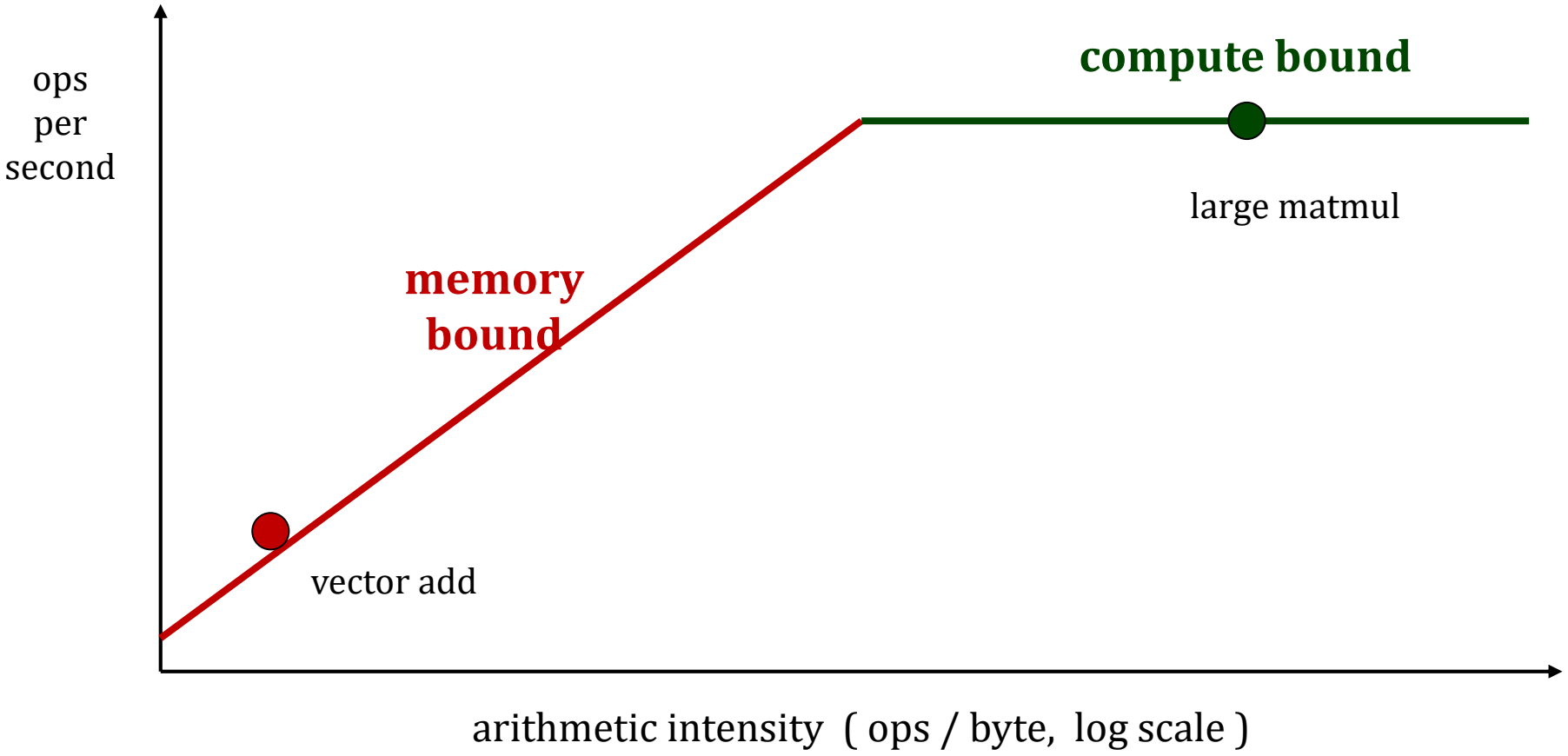
1 add per 12 bytes → 0.08 ops / byte

Matrix multiply, N × N, 16-bit:

$2N^3$ ops per $6N^2$ bytes → N / 3 ops / byte

N = 4096: about **1365** ops / byte

The Roofline



H100 ridge point: ~ 989 TFLOP/s ÷ 3.35 TB/s ≈ **300** ops/byte

Tensor Cores

hardware built for one operation

CUDA Core vs Tensor Core

CUDA core

$d = a \times b + c$

one scalar FMA per instruction

Tensor core

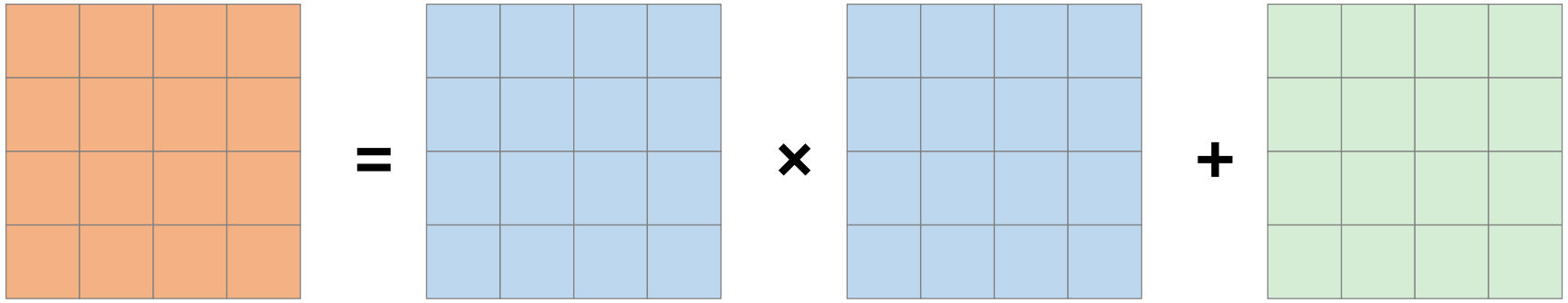
$D = A \times B + C$
on 4×4 matrices

64 FMAs per operation (Volta)

Same fetch, same decode, same register traffic

64 times the arithmetic

Inside One Tensor Core Operation

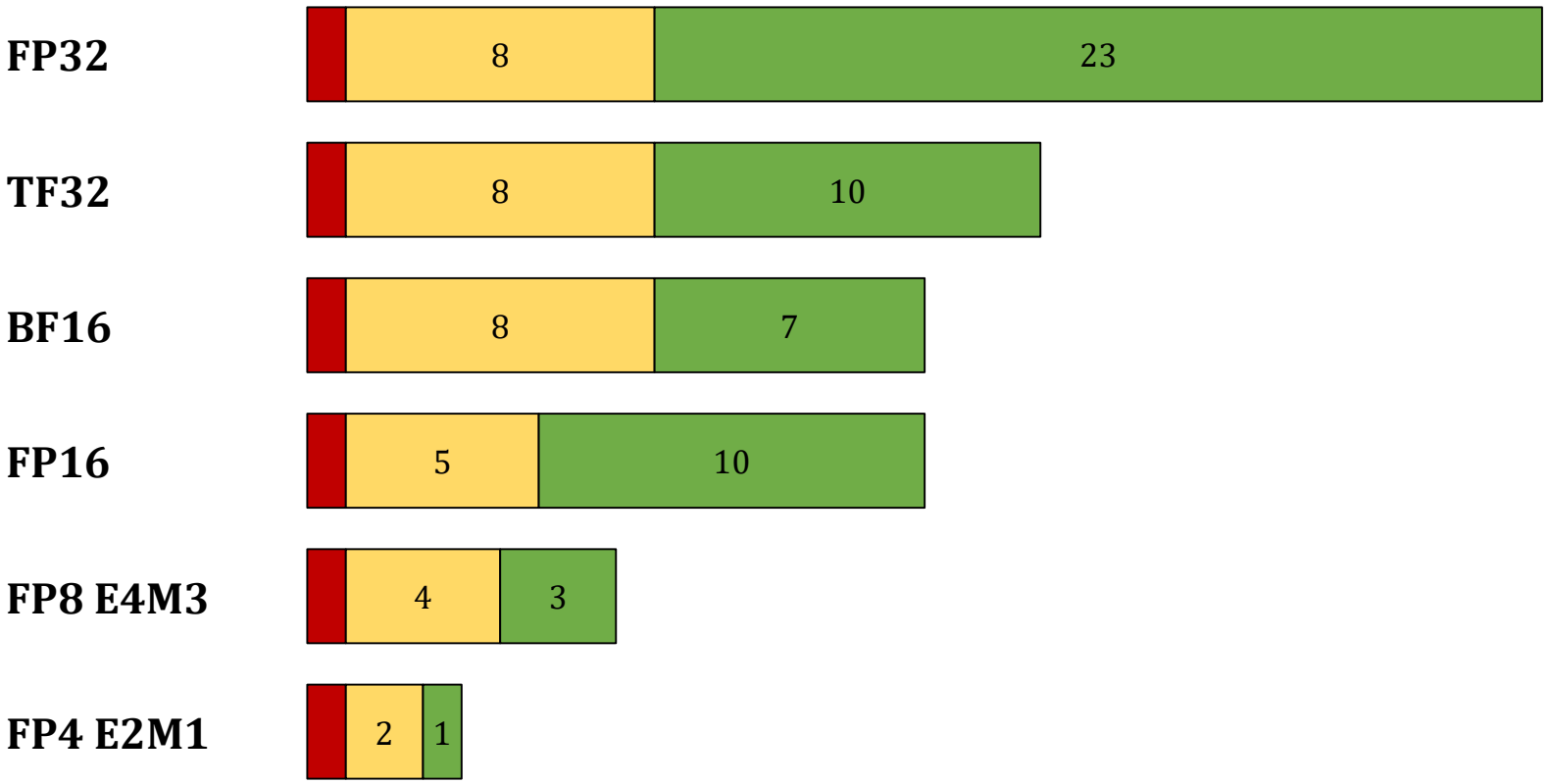


A and B: low precision (FP16, BF16, FP8 ...)

C and D: higher precision accumulator (FP32)

Inside: 64 multipliers feeding adder trees

Number Formats



red = sign amber = exponent bits (range) green = mantissa bits (precision)

Fewer bits: less memory, smaller multipliers

Why Smaller Is So Much Cheaper

An array multiplier for m-bit mantissas

has about $\mathbf{m^2}$ partial-product cells

FP32	24×24	= 576
BF16	8×8	= 64
FP8 E4M3	4×4	= 16

Roughly **9 ×** smaller going from FP32 to BF16

Very Small Numbers Need a Scale

FP4 has only 16 patterns: ±0, 0.5, 1, 1.5, 2, 3, 4, 6

Real values span a much wider range

Block scaling: a small group of values
shares one scale factor

Blackwell NVFP4: every 16 values share
one 8-bit scale

The Transformer Engine

Hopper, 2022

Hardware and software that track the range
of each layer's values as training runs

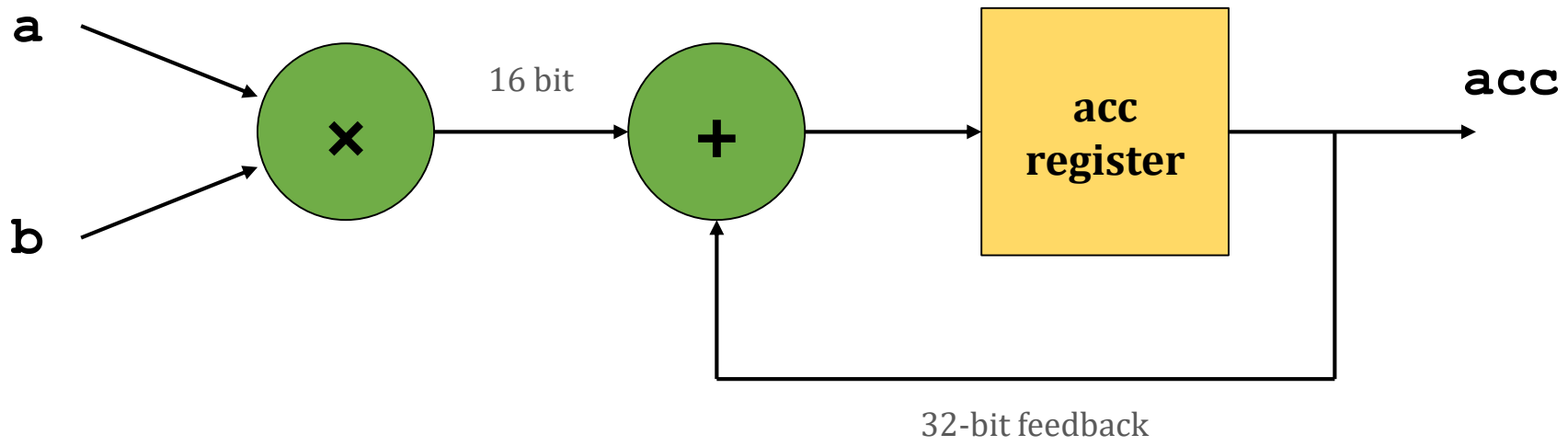
Choose FP8 where it is safe,
16-bit where it is not

Scale values so they fit the small format

Build a MAC

in SystemVerilog

MAC Unit



8-bit × 8-bit → 16-bit product

Accumulate in 32 bits so the sum does not overflow

MAC in SystemVerilog

```
module mac #(
    parameter int AW = 8,        // input width
    parameter int CW = 32        // accumulator width
) (
    input  logic                 clk, rst_n,
    input  logic                 clear, en,
    input  logic signed [AW-1:0] a, b,
    output logic signed [CW-1:0] acc
);
    logic signed [2*AW-1:0] prod;
    assign prod = a * b;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n)     acc <= '0;
        else if (clear) acc <= '0;
        else if (en)        acc <= acc + CW'(prod);
endmodule
```

Testing the MAC

```
module tb_mac;
    logic clk = 0, rst_n = 0, clear = 0, en = 0;
    logic signed [7:0]  a, b;
    logic signed [31:0] acc;
    int expected = 0;

    mac dut (.*);
    always #5 clk = ~clk;

    initial begin
        @(negedge clk) rst_n = 1;
        repeat (100) begin
            @(negedge clk);
            a = $urandom;  b = $urandom;  en = 1;
            expected += a * b;
        end
        @(negedge clk) en = 0;
        if (acc == expected) $display("PASS  acc = %0d", acc);
        else $display("FAIL  acc = %0d  expected %0d", acc, expected);
        $finish;
    end
endmodule
```

Systolic Arrays

a grid of MACs that passes data along

What Is a Systolic Array?

A grid of identical **processing elements** (PEs)

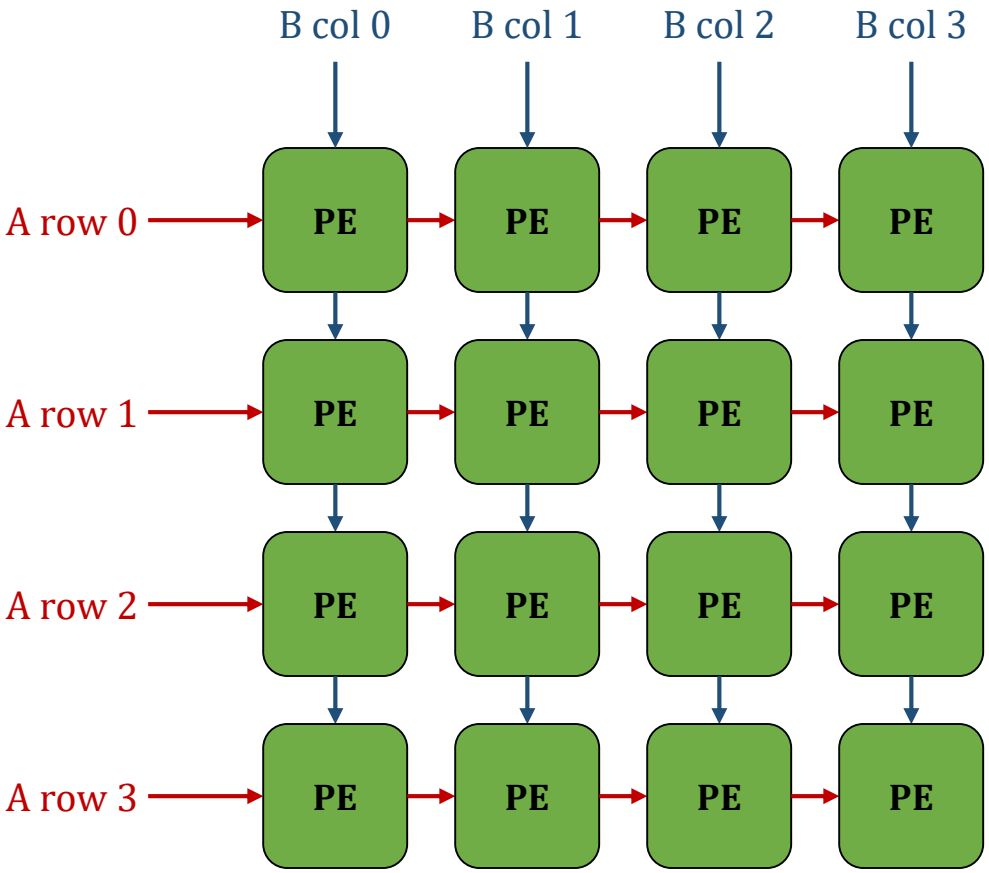
Each PE: one MAC, plus registers

Data **pulses** through the grid, neighbour to neighbour

like blood through the heart: systole

Each value fetched once, used by a whole row or column

A 4 × 4 Systolic Array



A flows right

B flows down

Each PE keeps its own

C[r][c]

The Processing Element

```
module pe #(parameter int AW = 8, CW = 32) (
    input  logic                  clk, rst_n, clear,
    input  logic signed [AW-1:0] a_in,      // from the left
    input  logic signed [AW-1:0] b_in,      // from above
    output logic signed [AW-1:0] a_out, // to the right
    output logic signed [AW-1:0] b_out, // downward
    output logic signed [CW-1:0] c
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            a_out <= '0;  b_out <= '0;  c <= '0;
        end else begin
            a_out <= a_in;              // pass A along
            b_out <= b_in;              // pass B along
            c     <= clear ? '0 : c + a_in * b_in;
        end
endmodule
```

Building the Array with generate

```
module systolic #(parameter int N = 4, AW = 8, CW = 32) (
    input  logic                 clk, rst_n, clear,
    input  logic signed [AW-1:0] a_left [N],
    input  logic signed [AW-1:0] b_top  [N],
    output logic signed [CW-1:0] c_out  [N][N]
);
    logic signed [AW-1:0] a_h [N][N+1]; // horizontal wires
    logic signed [AW-1:0] b_v [N+1][N]; // vertical wires

    for (genvar i = 0; i < N; i++) begin : g_edge
        assign a_h[i][0] = a_left[i];
        assign b_v[0][i] = b_top[i];
    end

    for (genvar r = 0; r < N; r++) begin : g_row
        for (genvar c = 0; c < N; c++) begin : g_col
            pe #(.AW(AW), .CW(CW)) u_pe (
                .clk, .rst_n, .clear,
                .a_in(a_h[r][c]),  .a_out(a_h[r][c+1]),
                .b_in(b_v[r][c]),  .b_out(b_v[r+1][c]),
                .c(c_out[r][c]));
        end
    end
endmodule
```

Feeding the Array: Skew

Row r of A enters r cycles late; column c of B enters c cycles late

cycle	0	1	2	3	4	5	6
`a_left[0]`	A00	A01	A02	A03			
`a_left[1]`		A10	A11	A12	A13		
`a_left[2]`			A20	A21	A22	A23	
`a_left[3]`				A30	A31	A32	A33

PE (r, c) sees A[r][k] and B[k][c] at cycle r + c + k

Last result ready after **3N – 2** cycles

Output vs Weight Stationary

Output stationary (ours)

each PE keeps one C value

A and B both move

Weight stationary (Google TPU)

each PE holds one **weight**, preloaded

inputs move, partial sums move

Choose by which data you most want to reuse

Two Ways to Build an AI Chip

GPU

Many **small** tensor cores inside each SM

Surrounded by general programmable cores

Runs graphics, HPC, AI

Flexible

Systolic accelerator

One or a few **huge** arrays

Little general-purpose logic

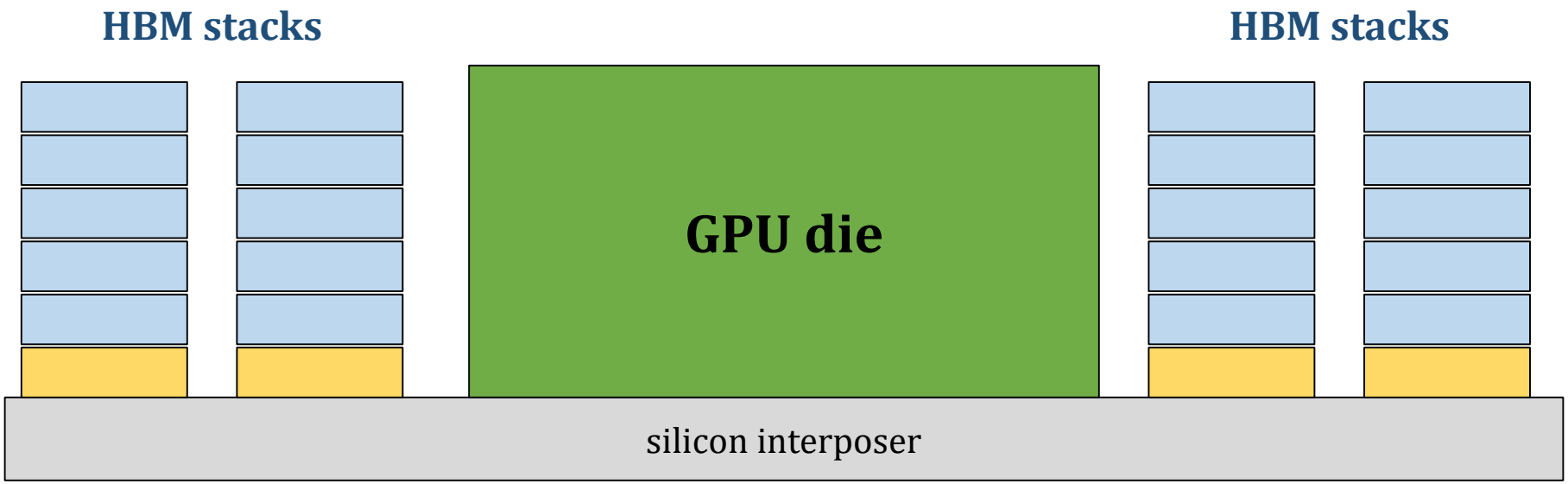
Built mostly for AI

More MACs per mm^2

Beyond the Core

memory and scale

High Bandwidth Memory



DRAM dies **stacked** and wired vertically, millimetres from the GPU

Very wide bus: thousands of wires instead of 64

Scaling Out

The biggest models do not fit on one GPU

NVLink connects GPUs directly

- Blackwell: 1.8 TB/s per GPU
- Rubin: 3.6 TB/s per GPU

Rack-scale systems: 72 GPUs act as one

Now the interconnect is part of the architecture

Using Tensor Cores

from PyTorch down to the hardware

PyTorch Mixed Precision

```
import torch

model = MyNet().cuda()
opt   = torch.optim.AdamW(model.parameters())

for x, y in loader:
    x, y = x.cuda(), y.cuda()
    with torch.autocast("cuda", dtype=torch.bfloat16):
        loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()
    opt.zero_grad()
```

Matrix multiplies in BF16 on tensor cores; weights kept in FP32

Tensor Cores from CUDA: WMMA

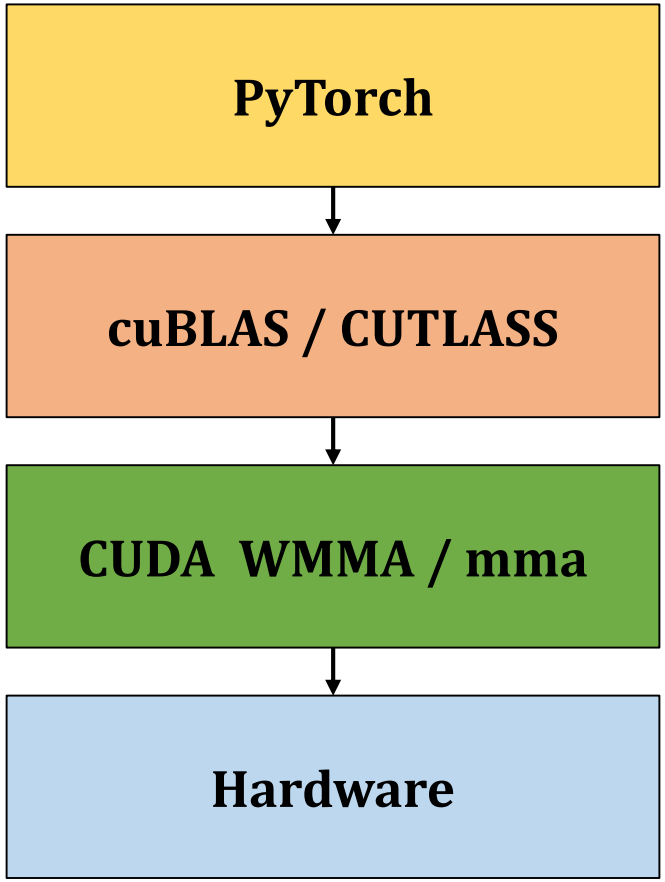
```
#include <mma.h>
using namespace nvcuda;

__global__ void wmma_16x16(const half *a, const half *b, float *c)
{
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> fa;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> fb;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> fc;

    wmma::fill_fragment(fc, 0.0f);
    wmma::load_matrix_sync(fa, a, 16);
    wmma::load_matrix_sync(fb, b, 16);
    wmma::mma_sync(fc, fa, fb, fc);     // D = A x B + C
    wmma::store_matrix_sync(c, fc, 16, wmma::mem_row_major);
}

wmma_16x16<<<1, 32>>>(d_a, d_b, d_c);   // one warp
```

One Operation, Four Levels




```
c = a @ b
```

```
tiled GEMM kernel
```

```
mma_sync( D, A, B, C )
```

```
array of MACs:  acc <= acc + a × b
```

You wrote the bottom level yourself

Key Ideas from Part 3

AI is mostly matrix multiply, which is mostly **MAC**

Tensor cores: a whole matrix operation per instruction

Fewer bits → smaller multipliers, less bandwidth

Systolic arrays reuse each value across a row or column

Bandwidth and interconnect matter as much as ALUs

The Whole Story

Part 1 throughput beats latency for parallel work

Part 2 CUDA: write it for one thread,
run it on millions

Part 3 the AI GPU is a sea of MACs,
fed by very wide memory

Every level of it starts as an HDL description

Let's Build an Array

Exercise: 4 × 4 Systolic Matrix Multiply

1. Type in pe and systolic with N = 4
2. Write a testbench that skews the rows of A
   and the columns of B as on the timing slide
3. Compare c_out with a software matrix multiply
   for 20 random signed 8-bit matrices
4. Count the cycles. Do you get 3N − 2 ?
5. Set N = 8. What changes in your RTL? (nothing!)